

# Cloud Computing Lab Guide

Comprehensive Tutorial: Virtual Machine Provisioning with Terraform (KVM/QEMU, OCI, VirtualBox)

| COURSE MODULE                | CORE TOOLS                | TARGET OS        |
|------------------------------|---------------------------|------------------|
| Infrastructure as Code (IaC) | Terraform, KVM, OCI, VBox | Ubuntu 22.04 LTS |

## 1. Lab Overview & Prerequisites

This hands-on guide provides complete step-by-step instructions and code manifests for provisioning Virtual Machines across various hypervisors and cloud platforms using HashiCorp Terraform.

### Core Learning Objectives

Configure environment dependencies, write reusable HCL (HashiCorp Configuration Language) scripts, utilize Cloud-Init for dynamic VM contextualization, and deploy single and multi-node clusters.

| Target Environment | Provider Name         | Driver / Core Engine | Primary Use Case                                    |
|--------------------|-----------------------|----------------------|-----------------------------------------------------|
| Local QEMU/KVM     | dmacvicar/libvirt     | libvirt / QEMU       | High-performance local virtualization & K8s testing |
| Oracle Cloud (OCI) | oracle/oci            | OCI Compute Service  | Production cloud infrastructure deployment          |
| Oracle VirtualBox  | terra-farm/virtualbox | VBoxManage CLI       | Cross-platform desktop VM automation                |

## 2. Lab Setup: Environment & Package Installation

Execute the following script on your Ubuntu 22.04 LTS host/VM to prepare the system with required build packages, Python dependencies, and the official HashiCorp repository.

```
# 1. Update system packages & install dependencies
sudo apt-get update -y && sudo apt-get upgrade -y
sudo apt-get install -y curl unzip gnupg software-properties-common git python3-pip python3-venv

# 2. Add HashiCorp Official GPG Key & Repository
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/
hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/
hashicorp.list

# 3. Install Terraform
sudo apt-get update && sudo apt-get install terraform -y

# 4. Verify Installation
terraform -version
```

## 3. QEMU / KVM Virtual Machine Provisioning (Libvirt)

### 3.1 Host Hypervisor Preparation

Ensure KVM and `libvirtd` packages are enabled and your current user is added to the `libvirt` group:

```
sudo apt-get install -y qemu-kvm libvirt-daemon-system libvirt-clients bridge-utils virt-manager  
sudo systemctl enable --now libvirtd  
sudo usermod -aG libvirt $USER && newgrp libvirt
```

### 3.2 Multi-Node QEMU Cluster Manifest ( `main.tf` )

The following configuration deploys a 3-node Kubernetes-ready cluster (1 Control Plane, 2 Workers) using Cloud-Init for automated SSH setup.

```

terraform {
  required_version = ">= 1.0.0"
  required_providers {
    libvirt = {
      source = "dmacvicar/libvirt"
      version = "~> 0.7.0"
    }
  }
}

provider "libvirt" {
  uri = "qemu:///system"
}

# Node Configuration
locals {
  cluster_nodes = {
    "k8s-control-plane" = { role = "control-plane", vcpu = 2, memory = 4096, disk_gb = 20 }
    "k8s-worker-1"      = { role = "worker",          vcpu = 2, memory = 2048, disk_gb = 15 }
    "k8s-worker-2"      = { role = "worker",          vcpu = 2, memory = 2048, disk_gb = 15 }
  }
}

# Base Cloud Image
resource "libvirt_volume" "ubuntu_base" {
  name     = "ubuntu-22.04-base.qcow2"
  pool     = "default"
  source   = "https://cloud-images.ubuntu.com/releases/22.04/release/ubuntu-22.04-server-cloudimg-amd64.img"
  format   = "qcow2"
}

# Individual VM Disks
resource "libvirt_volume" "vm_disks" {
  for_each = local.cluster_nodes
  name     = "${each.key}-disk.qcow2"
  base_volume_id = libvirt_volume.ubuntu_base.id
  pool     = "default"
  size     = each.value.disk_gb * 1073741824
}

# Cloud-Init Disk
resource "libvirt_cloudinit_disk" "cluster_init" {
  for_each = local.cluster_nodes
  name     = "${each.key}-init.iso"
  pool     = "default"
  user_data = <<-EOF
#cloud-config
hostname: ${each.key}
fqdn: ${each.key}.k8s.local
manage_etc_hosts: true
users:
  - name: ubuntu
    sudo: ALL=(ALL) NOPASSWD:ALL
    groups: users, admin
    home: /home/ubuntu
    shell: /bin/bash
    ssh_authorized_keys:
      - ${file("${~}/.ssh/id_rsa.pub")}
ssh_pwauth: true
EOF
}

```

```

disable_root: false
EOF
}

# Dedicated NAT Network
resource "libvirt_network" "cluster_net" {
  name      = "qemu_cluster_net"
  mode      = "nat"
  domain    = "k8s.local"
  addresses = ["192.168.100.0/24"]
  dhcp { enabled = true }
  dns   { enabled = true }
}

# Domain Provisioning
resource "libvirt_domain" "cluster_vms" {
  for_each = local.cluster_nodes
  name      = each.key
  memory    = each.value.memory
  vcpu      = each.value.vcpu

  cloudinit = libvirt_cloudinit_disk.cluster_init[each.key].id

  network_interface {
    network_id      = libvirt_network.cluster_net.id
    wait_for_lease = true
  }

  disk { volume_id = libvirt_volume.vm_disks[each.key].id }

  console {
    type      = "pty"
    target_port = "0"
    target_type = "serial"
  }
}

output "cluster_summary" {
  value = {
    for name, vm in libvirt_domain.cluster_vms : name => {
      role = local.cluster_nodes[name].role
      ip    = vm.network_interface[0].addresses[0]
      ram   = "${local.cluster_nodes[name].memory} MB"
    }
  }
}

```

## 4. Oracle Cloud Infrastructure (OCI) Deployment

This manifest creates a Virtual Cloud Network (VCN), public subnet, internet gateway, and provisions an Ubuntu VM instance on Oracle Cloud.

```

terraform {
  required_providers {
    oci = {
      source = "oracle/oci"
      version = "~> 5.0"
    }
  }
}

provider "oci" {
  tenancy_ocid = var.tenancy_ocid
  user_ocid    = var.user_ocid
  fingerprint  = var.fingerprint
  private_key_path = var.private_key_path
  region       = var.region
}

resource "oci_core_vcn" "lab_vcn" {
  compartment_id = var.compartment_ocid
  cidr_blocks    = ["10.0.0.0/16"]
  display_name   = "cloud-lab-vcn"
}

resource "oci_core_internet_gateway" "lab_ig" {
  compartment_id = var.compartment_ocid
  vcn_id         = oci_core_vcn.lab_vcn.id
  display_name   = "cloud-lab-ig"
  enabled        = true
}

resource "oci_core_subnet" "lab_subnet" {
  compartment_id = var.compartment_ocid
  vcn_id         = oci_core_vcn.lab_vcn.id
  cidr_block     = "10.0.1.0/24"
  display_name   = "cloud-lab-subnet"
}

resource "oci_core_instance" "ubuntu_vm" {
  compartment_id = var.compartment_ocid
  availability_domain = data.oci_identity_availability_domain.ad.name
  display_name     = "oci-ubuntu-vm"
  shape            = "VM.Standard.A1.Flex"

  shape_config {
    ocpus      = 1
    memory_in_gbs = 6
  }

  create_vnic_details {
    subnet_id = oci_core_subnet.lab_subnet.id
    assign_public_ip = true
  }

  metadata = {
    ssh_authorized_keys = file(var.ssh_public_key_path)
  }
}

```

## 5. VirtualBox VM Provisioning

To provision VirtualBox instances on desktop or laptop environments, use the `terra-farm/virtualbox` provider:

```
terraform {
  required_providers {
    virtualbox = {
      source = "terra-farm/virtualbox"
      version = "~> 0.2.2"
    }
  }
}

resource "virtualbox_vm" "node" {
  name      = "vbox-ubuntu-node"
  image     = "https://app.vagrantup.com/ubuntu/boxes/bionic64/versions/20230607.0.1/providers/virtualbox.box"
  cpus      = 2
  memory    = "2048 mib"

  network_adapter {
    type          = "nat"
    host_interface = "vboxnet0"
  }
}
```

## 6. Execution & Verification Workflow

1. Initialize working directory: `terraform init`
2. Review infrastructure plan: `terraform plan`
3. Deploy virtual machines: `terraform apply -auto-approve`
4. Access VM over SSH: `ssh ubuntu@<VM_IP_ADDRESS>`
5. Teardown infrastructure when complete: `terraform destroy -auto-approve`